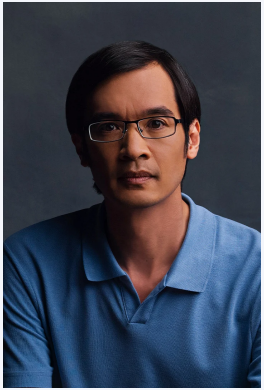
Algebraic Reasoning Distiller

Harnessing Multi-Agent Collaboration for Mathematical Problem Solving

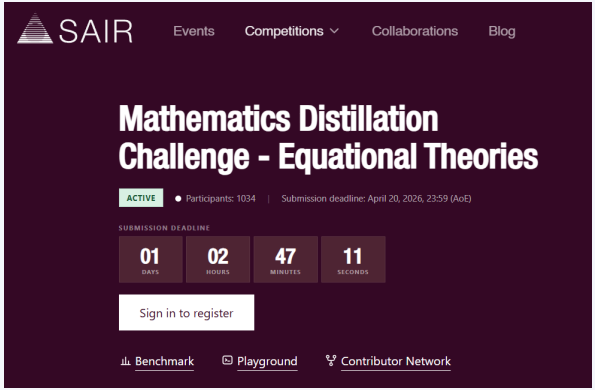
SAIR Foundation — Mathematics Distillation Challenge, Stage 1: Equational Theories

Joaquín Mir Macías · José Andrés Ridruejo Tuñón · Javier Ríos Montes · Miguel Montes Lorenzo

Deep Generative Models · Master in Artificial Intelligence · U.P.Comillas 2026



Terence Tao, associated with the SAIR Foundation and the broader motivation behind formal mathematical reasoning.



SAIR Mathematics Distillation Challenge landing page: *Equational Theories*.

The gap in LLM mathematical reasoning:

- LLMs solve math by **pattern matching**, not symbolic understanding
- Consistent errors in algebraic manipulation and abstract inference
- **Pretraining** on math corpora is prohibitively expensive
- Naive **fine-tuning** produces narrow experts
- Even frontier models show weaknesses in abstract algebra

Our approach

Symbolic reasoning + **LLM distillation** via multi-agent offline computation, within strict competition constraints.

Mathematical formulation:

A **magma** is a set M with a binary operation $* : M \times M \to M$ *(no axioms assumed)*.

Given E_1, E_2 built with $*$, decide:

$$\boxed{E_1 \overset{?}{\Longrightarrow} E_2 \text{ in every magma}}$$

Example:

E_1: $x * y = z * (w * (u * u))$

E_2: $x * (y * y) = (z * w) * z$

Answer: TRUE every magma satisfying E_1 also satisfies E_2

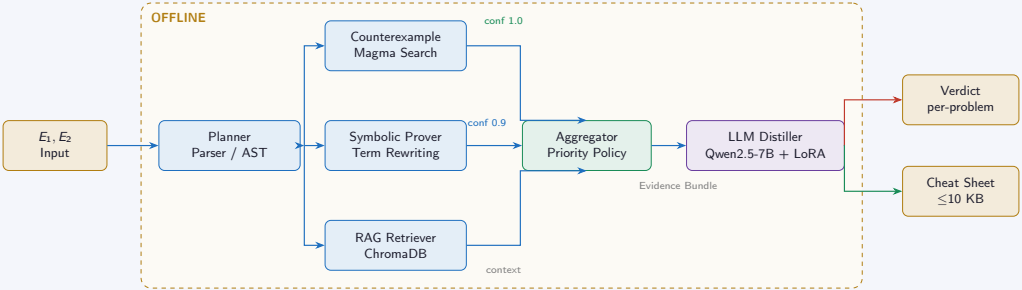
Competition constraints:

$\leq$10 KB	Static cheat sheet — only resource allowed
$\leq$10 min	Time budget per problem
\$0.01	Compute budget per problem
0 tools	No external tools at evaluation time

Key insight

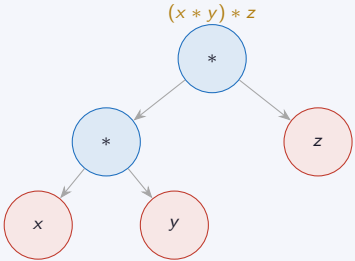
All heavy compute happens **offline**. At test time: cheat sheet + one LLM call.

System Architecture | ARD — Algebraic Reasoning Distiller



Recursive-descent parser

Input string → nested tuple AST:




```
("mul", ("mul", ("var","x"), ("var","y")),
        ("var","z"))
```

Why ASTs?

- Canonical representation **shared by all agents**
- Enables structural comparison: alpha-equivalence, substitution
- Decouples **surface syntax** from mathematical content
- Variables: `("var","x")`
  Operations: `("mul", left, right)`

Design principle

Every agent receives the same AST — errors propagate cleanly and the pipeline **never crashes** on malformed input.

Counterexample Generator

Find magma M: E_1 holds, E_2 fails.

Order 2: exhaustive ($2^4 = 16$ magmas)

↓

Order 3: exhaustive ($3^9 = 19\,683$ magmas)

↓

Order 4: random sample ($20\,000$ / 4^{16})

↓

Witness found?
Hard-FALSE, conf = 1.0
No witness ⇒ inconclusive

Symbolic Prover

Three strategies, applied in order:

1. Alpha-equivalence
$E_1 \equiv E_2$ up to variable renaming

↓

2. Substitution instance
substituting terms in E_1 yields E_2

↓

3. One-step term rewriting
$E_1 \xrightarrow{\text{rule}} E_2$

Proof found ⇒ Hard-TRUE, conf = 0.9
Conservative: negative ≠ FALSE

RAG Retriever

E_1, E_2 query string

↓

`all-MiniLM-L6-v2`
embedding model

↓

ChromaDB vector store
4 694 canonical equations

↓

Top k=5 by cosine similarity
injected as LLM context

Degrades gracefully if index is absent.

Aggregator — Priority chain

Counterexample found? conf 1.0
⇒ Hard-FALSE, pipeline stops

↓

Symbolic proof found? conf 0.9
⇒ Hard-TRUE, pipeline stops

↓

Neither conf 0.0
⇒ delegate to LLM Distiller

Negative result ≠ falsity: the prover is deliberately conservative.

Model stack:

Qwen2.5-7B-Instruct (base)
4-bit NF4 quantization + BF16 compute

↓

LoRA Adapters
rank r=16, α=32, dropout 0.05
All Attention & MLP projections
≈42M trainable / 7.6B total

↓

Structured output
`<think>...</think>`
`<answer>VERDICT: TRUE|FALSE...`

Two inference modes:

Per-problem mode

Input: EvidenceBundle + RAG context
Output: chain-of-thought + VERDICT

Cheat-sheet synthesis mode

Input: cluster of similar problems
Output: $\leq$1 200-byte lemma block
`VERDICT` tokens **banned** in synthesis prompt

Why Qwen2.5-7B?

Fits in 4-bit on a single GPU · satisfies the $0.01/problem budget · strong mathematical baseline

Source	#	Notes
SAIR judge repo	20	canonical labelled examples
Playground hard3	72	manually curated playground set
Competition dumps	1 669	`hard.txt`, `hard2.txt`, `hard3.txt`, `normales.txt`
Retrieval index (RAG)	4 694	equations embedded in ChromaDB for retrieval only
Labelled corpus total	**1 761**	training/evaluation corpus

Train / Eval split:

80% train — 1 409 problems

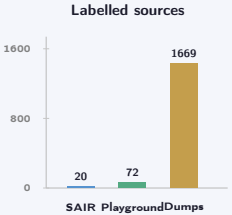
20% eval — 352 problems

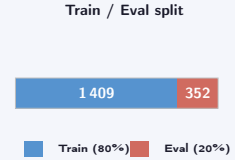
SFT filtering:

Problems where pipeline verdict $\neq$ ground truth are excluded from SFT.

Approximately 1 330 clean examples remain.

RAG index size is reported separately from the labelled corpus.





Separate retrieval index: 4 694 equations

EvidenceBundle → structured completion:

```
<think>
  E1:  x*y = y*x
  Substitution matches by renaming.
  Confidence:  0.9 (TRUE path).
</think>

<answer>
  VERDICT: TRUE
  REASONING: alpha-equivalence
  PROOF: direct substitution
</answer>
```

Hyperparameters:

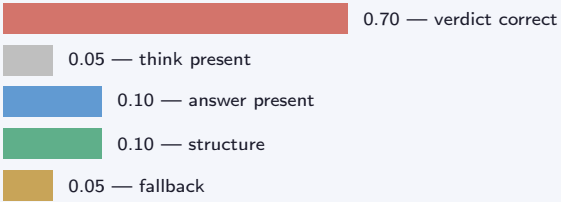
Epochs	2
Learning rate	2×10^{-4}
Batch size	2
Grad. accumulation	2
Scheduler	cosine
Precision	BF16 mixed

Goal

Teach the model *format compliance* first. GRPO will then optimise verdict correctness.

Reward function:

$$R = \underbrace{0.70 \cdot 1[\text{verdict correct}]}_{\text{dominant}} + 0.10 \cdot 1[\texttt{<answer>}] + 0.10 \cdot 1[\text{struct.}] + 0.05 \cdot 1[\texttt{<think>}] + 0.05 \cdot 1[\text{fallback}]$$



GRPO algorithm:

Generate G=2 completions per problem

↓

Score with SAIR judge oracle
normalise advantages within group

↓

Policy gradient update
KL penalty β=0.02 vs. SFT ref.

↓

1 epoch · lr = 10^{-5} · max 512 tokens
gradient clip 1.0

Pipeline:

Cluster by structural signature
`vars{n}_m{d1}_n{d2}`

↓

Balanced selection:
equal #TRUE-majority & #FALSE-majority

↓

Synthesis prompt bans
`VERDICT:`, `<answer>`, `<think>`

↓

Greedy pack $\leq 1\,200$ bytes/cluster
Total budget: 10 000 bytes

v1 failure & v2 fix:

v1 — Collapsed to 20% accuracy

FALSE-dominated clusters leaked verdict tokens into synthesis $\Rightarrow$ systematic bias toward FALSE

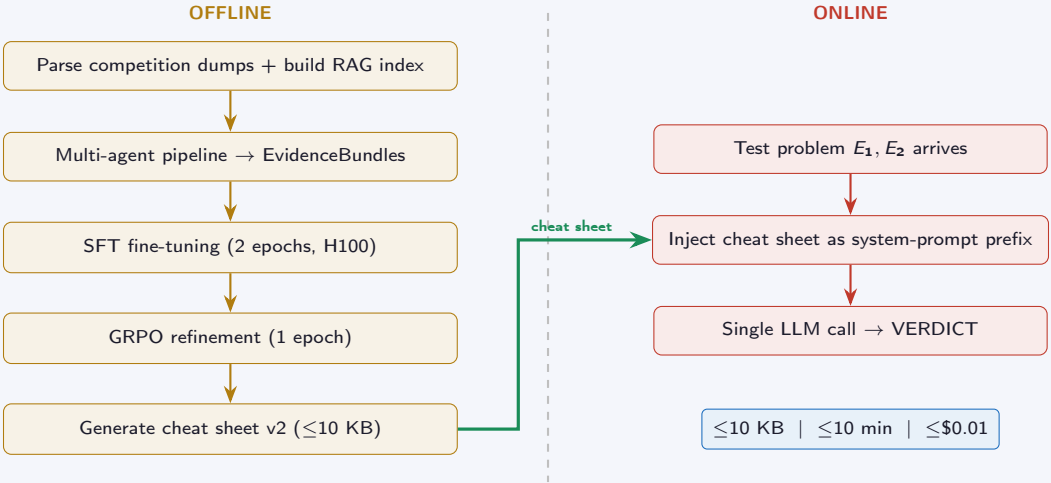
v2 — Balanced clustering

Enforce equal TRUE/FALSE cluster counts + dedicated synthesis prompt forbids verdict tokens

Preamble entry (priority 999):

```
HOW TO USE THIS CHEAT SHEET
Read lemma packs and apply the
matching pattern to derive a verdict.
```

Distill, Then Deploy | Offline training vs. online evaluation



Performance table:

Configuration	Acc.	Parse
Base model (no fine-tuning)	0.50	0.75
Symbolic agents only	0.50	—
SFT only (20-problem corpus)	0.75	1.00
SFT + GRPO (20-problem corpus)	0.75	1.00
+ cheat sheet **v1**	**0.20**	1.00
+ cheat sheet **v2**	**in prog.**	1.00

Training infrastructure: DGX cluster (8× NVIDIA H100)

[IMAGE PLACEHOLDER]

Accuracy progression bar chart:

Base → SFT → SFT+GRPO → v1 (regression) → v2

Annotate v1 bar with *"FALSE bias"*

matplotlib, light background
AccentRed / AccentGreen bars

Conclusions & Future Work

ARD — Algebraic Reasoning Distiller

Key contributions:

- ✓ Multi-agent pipeline: **symbolic** + **neural** reasoning
- ✓ SFT → GRPO within **severe resource constraints**
- ✓ Identified & fixed cheat-sheet **verdict-bias** (v1 → v2)
- ✓ Reproducible on DGX (Docker + `uv`)

Lessons learned:

Data **balance** is critical. Synthesis prompt must *decouple* generation from judgment. Graceful degradation enables robust deployment.

Future work:

- ▷ Complete cheat sheet **v2** on full 1 761-problem corpus
- ▷ Order-4 **exhaustive** magma search
- ▷ Distil into **smaller models** (<3B params)
- ▷ Iterative cheat-sheet refinement via GRPO loop

Repository: MIT Licensed
github.com/joaquinmaciias/Algebraic-Reasoning-Distiller.git